GUÍA GENERAL DE PRÁCTICAS

Equipo, entorno, convenios y evaluación

DOCUMENTO	ASIGNATURA	TITULACIÓN	CENTRO
Del alumnado	**Arq. de Computadores II**	**Grado en Ing. Informática · 2.º**	**ESEI · Universidade de Vigo**

Contenido

1. Para qué sirve este documento 3

2. Las siete prácticas de un vistazo 3

2.1 El hilo conductor 4

3. El equipo 5

3.1 La placa 5

3.2 Los pines, iguales en todas las prácticas 5

3.3 El módulo TM1638 «LED&KEY» 6

4. El entorno de trabajo 7

4.1 Dos avisos antes de empezar 7

4.2 Cómo probar un programa ya compilado 7

5. El reloj: 84 MHz en todas las prácticas 8

5.1 Por qué esto importa más de lo que parece 8

6. Convenios comunes a las siete prácticas 9

6.1 El contrato USER CODE 9

6.2 Esperar sin bloquear 10

6.3 La palabra volatile 10

6.4 Nombres 11

7. La biblioteca del TM1638 11

7.1 Un display de siete segmentos no puede con todas las letras 12

8. Trampas que se repiten 13

8.1 La trampa que no se ve simulando 14

9. Evaluación 14

9.1 Consejos para la memoria 15

10. Qué recibes de cada práctica 15

10.1 Qué se entrega 16

1. Para qué sirve este documento

Las siete prácticas de la asignatura tienen cada una su enunciado, y ninguno repite lo que ya explicó el anterior. Eso las hace cortas, pero deja fuera lo que **es común a todas** y lo que solo se ve mirando el conjunto: el equipo, el entorno, los convenios de código, cómo se evalúa y las trampas que reaparecen práctica tras práctica.

Este documento recoge eso. **Se lee una vez, al principio de curso**, y después se vuelve a él cuando algo no encaja. No sustituye a ningún enunciado: los complementa.

2. Las siete prácticas de un vistazo

	TÍTULO	QUÉ INTRODUCE	QUÉ DA POR SABIDO
P1	Primer proyecto	El fichero de configuración, CubeMX, el reloj del sistema y el contrato `USER CODE`. Entradas y salidas digitales.	— (es la primera)
P2	Puerta de garaje	El módulo TM1638. Máquina de estados. Antirrebote. Cadencia no bloqueante con `HAL_GetTick()`.	P1
P3	Cronómetro	Base de tiempo sin deriva. Por qué > y >= no son intercambiables al comparar tiempos.	P1, P2
P4	Interrupciones	EXTI, SysTick y TIM2. `volatile`. El límite de 24 bits del SysTick.	P1 a P3
P5	Conversión A/D	El ADC en sus tres modos: consulta, interrupción y DMA. Sensor de temperatura y referencia interna.	P1 a P4

	TÍTULO	QUÉ INTRODUCE	QUÉ DA POR SABIDO
P6	Ordenador de a bordo	*Proyecto final, sesión 1.* Integración: simulación física, varias pantallas, unidades derivadas.	todo lo anterior
P7	Semáforo de salida	*Proyecto final, sesión 2.* Medida de tiempos de reacción capturando en la interrupción. Azar sin generador aleatorio.	todo lo anterior

> **P6 y P7 no introducen materia nueva.** Son el proyecto final: dos sesiones para juntar lo aprendido en algo que funciona de principio a fin. Quien lleve al día P1 a P5 puede hacerlas sin material adicional.

2.1 El hilo conductor

La secuencia no es una lista de temas sueltos: cada práctica necesita la anterior, y el salto entre dos consecutivas es siempre **una sola idea nueva**.

- **P1 a P2.** En P1 el programa espera parándose. En P2 eso ya no vale, porque hay que atender el pulsador mientras la puerta se mueve: aparece la cadencia no bloqueante.
- **P2 a P3.** La cadencia de P2 funciona, pero acumula deriva. P3 la mide, la explica y la corrige.
- **P3 a P4.** Hasta aquí todo se consulta desde el bucle. P4 le da la vuelta: que sea el periférico quien avise.
- **P4 a P5.** Con las interrupciones ya dominadas, el ADC se puede ver en sus tres modos y compararlos de verdad.
- **P5 a P6 y P7.** Nada nuevo. Se trata de integrar, y de que el resultado se pueda enseñar a alguien.

3. El equipo

3.1 La placa

DATO	VALOR
Placa	NUCLEO-F401RE
Microcontrolador	`STM32F401RET6`, ARM Cortex-M4 con unidad de coma flotante
Frecuencia	**84 MHz** en las siete prácticas (apartado 5)
Memoria	512 KB de Flash, 96 KB de RAM
Programador	ST-LINK integrado. Se conecta con un único cable USB
Manual de referencia	RM0368. **No** el RM0090, que es de la familia F405 y F407

> El ST-LINK aparece en el ordenador como una **unidad extraíble** llamada `NODE_F401RE`. Copiar ahí un fichero `.bin` graba la placa: no hace falta abrir el entorno para probar un programa ya compilado.

3.2 Los pines, iguales en todas las prácticas

Todos los proyectos usan la misma asignación. Si al montar el módulo se respeta esta tabla, cualquier programa del bloque funciona sin tocar nada.

PIN	NOMBRE	QUÉ ES	DÓNDE SE USA
PA5	`LD2`	LED verde de la placa	P1, P4
PC13	`B1`	Pulsador azul de usuario. **Activo a cero**	P1, P4, P5, P6, P7

PIN	NOMBRE	QUÉ ES	DÓNDE SE USA
PB4	`CLK`	Reloj del TM1638	P2 a P7
PB5	`DIO`	Dato del TM1638, bidireccional	P2 a P7
PA10	`STB`	Selección del TM1638	P2 a P7
PA1	`ADC1_IN1`	Potenciómetro	P5, P6
PA2 y **PA3**	USART2	Puerto serie del ST-LINK. Se configura solo	ninguna lo usa

> **`B1` es activo a cero**: pulsado se lee `GPIO_PIN_RESET`. Tenlo presente al elegir el flanco de una interrupción, porque con lógica invertida el flanco de **subida** es el momento de soltar el botón, no el de pulsarlo. Un programa que reaccione al flanco equivocado parece funcionar, pero siempre va un paso por detrás.

3.3 El módulo TM1638 «LED&KEY»

RECURSO	CANTIDAD	NOTA
Dígitos de siete segmentos	8	con punto decimal independiente
LEDs rojos	8	uno encima de cada dígito
Pulsadores	8	S1 a S8. Se leen los ocho de una vez
Pines de conexión	3	más alimentación y masa

Tres pines para veinticuatro elementos: el módulo lleva su propio controlador y se habla con él por un protocolo serie. La biblioteca que se te entrega lo resuelve, pero conviene

saber que **cada refresco cuesta tiempo**, y por eso ningún programa del bloque redibuja el display en cada vuelta del bucle.

4. El entorno de trabajo

HERRAMIENTA	VERSIÓN	PARA QUÉ
STM32CubeIDE	2.2.0	Editar, compilar y depurar
STM32CubeMX	6.18.1	Generar la configuración a partir del fichero `.ioc`
`arm-none-eabi-gcc`	14.3.1	El compilador. Va incluido en el entorno

4.1 Dos avisos antes de empezar

> **No guardes los proyectos en una ruta con tildes ni eñes.** El depurador no encontrará los ficheros fuente y, al poner un punto de ruptura, se parará mostrando ensamblador en vez de tu código en C. No da ningún error: simplemente no funciona como esperas. Una carpeta del tipo `C:\ACII` evita el problema.
>
> **Tampoco uses rutas muy largas.** Pasados los 260 caracteres que admite Windows, el compilador puede fallar **diciendo que todo ha ido bien**. Si el programa no cambia por más que lo modifiques, comprueba que el fichero `.elf` se haya creado de verdad, y acorta la ruta.

4.2 Cómo probar un programa ya compilado

1. Conecta la placa por USB. Aparecerá la unidad `NODE_F401RE`.
2. Arrastra el fichero `.bin` a esa unidad.
3. La placa se reinicia sola y arranca el programa nuevo. No hace falta nada más.

5. El reloj: 84 MHz en todas las prácticas

Los proyectos del bloque comparten exactamente la misma configuración de reloj. Es el máximo del F401, y lo genera CubeMX a partir del fichero de configuración: **no se escribe a mano**.

LA CADENA DEL PLL

```
HSI 16 MHz  /  PLLM 8   =   2 MHz
            x  PLLN 84  = 168 MHz
            /  PLLP 2   =  84 MHz   <- SYSCLK
```

SEÑAL	DIVISOR	FRECUENCIA	QUIÉN LA USA
SYSCLK y HCLK	÷1	**84 MHz**	el núcleo y la memoria
APB1 (PCLK1)	÷2	42 MHz	TIM2 y los temporizadores de baja velocidad
APB2 (PCLK2)	÷1	84 MHz	el ADC y los periféricos rápidos
Reloj del ADC	÷4 sobre APB2	21 MHz	por debajo del máximo de **36 MHz** del F401

> A 84 MHz la memoria Flash necesita **dos ciclos de espera** (`FLASH_LATENCY_2`). Con menos, el núcleo leería instrucciones antes de que la memoria las tenga listas. Lo ajusta CubeMX, pero conviene saber que existe: es la razón de que subir la frecuencia no sea solo cambiar un número.

5.1 Por qué esto importa más de lo que parece

Hay dos formas de esperar en un microcontrolador, y **solo una depende del reloj**:

FORMA DE ESPERAR	SI CAMBIA EL RELOJ	POR QUÉ
`HAL_Delay` y `HAL_GetTick()`	no cambia	`HAL_Init` ajusta el SysTick a un milisegundo sea cual sea la frecuencia
Contar vueltas de un bucle	**cambia**	cada vuelta cuesta un número fijo de ciclos, y los ciclos duran más o menos según el reloj
Prescaladores de temporizador	**cambia**	se calculan sobre la frecuencia del bus, no sobre milisegundos

> **Es un error que no avisa.** Si un proyecto corre a una frecuencia distinta de la que suponías, casi nada cambia, porque casi todo se apoya en `HAL_GetTick()`. Solo se rompe lo que cuenta ciclos: un prescalador calculado para 84 MHz y ejecutado a 16 da **5,25 veces** el periodo pretendido. Por eso, antes de dar por buena cualquier medida de tiempo, comprueba a qué frecuencia está configurado el proyecto.

6. Convenios comunes a las siete prácticas

6.1 El contrato USER CODE

CubeMX **reescribe** todo lo que hay fuera de las marcas `USER CODE` cada vez que se regenera el proyecto. Lo que está dentro se conserva. Es la regla más importante del bloque, y la que más código ha hecho perder a lo largo de los años.

```
/* USER CODE BEGIN 2 */
    /* aqui se escribe: esto sobrevive a la regeneracion */
/* USER CODE END 2 */
```

> Si algo tiene que ir **fuera** de las marcas por necesidad, apúntalo en la memoria: es una deuda que alguien pagará la próxima vez que abra CubeMX.

6.2 Esperar sin bloquear

A partir de P2, ningún programa usa `HAL_Delay` dentro del bucle principal. El patrón es siempre el mismo:

```
if (HAL_GetTick() - tAnterior >= PERIODO_MS) {
    tAnterior += PERIODO_MS;      /* sumar el periodo, NO reasignar */
    /* ...lo que toque hacer cada PERIODO_MS... */
}
```

DETALLE	POR QUÉ	SI SE HACE MAL
>= y no >	con > la condición no se cumple hasta el milisegundo siguiente	cada periodo dura uno de más: **un 10 % de error** con periodos de 10 ms
+= y no =	reasignar `HAL_GetTick()` descarta el tiempo sobrante	la deriva se acumula y el error crece sin parar

6.3 La palabra volatile

Toda variable que compartan una interrupción y el bucle principal se declara `volatile`. También el contador de un bucle de espera vacío, para que el compilador no lo borre.

> **`volatile` no hace nada por sí misma: le quita al compilador el permiso de optimizar.** Y ese permiso solo se ejerce con la optimización activada, así que un programa al que le falte puede funcionar perfectamente en *Debug* y romperse el día que alguien compile en *Release*, sin haber tocado una línea. Tampoco hace la variable atómica: para eso hacen falta otras cosas, que quedan fuera de la asignatura.

6.4 Nombres

- Identificadores en **inglés**; comentarios en **castellano** cuando hagan falta.
- Constantes en mayúsculas con `#define` y **con las unidades en el nombre**: `TICK_MS`, `RECORRIDO_CM`.
- Nada de números mágicos repartidos por el código.

7. La biblioteca del TM1638

Las prácticas de P2 en adelante comparten la misma biblioteca: dos ficheros, `TM1638.c` y `TM1638.h`, que se copian a cada proyecto. Van copiados y no enlazados para que cada proyecto se pueda comprimir y compilar por su cuenta, sin depender de nada externo.

No hay función de inicialización. Los puertos y pines se asignan a mano sobre la estructura, y después se limpia el display. Es lo que hacen los doce proyectos:

```
TM1638 TM;
TM.clk_port = CLK_GPIO_Port;  TM.clk_pin = CLK_Pin;
TM.stb_port = STB_GPIO_Port;  TM.stb_pin = STB_Pin;
TM.dio_port = DIO_GPIO_Port;  TM.dio_pin = DIO_Pin;
tm1638_DisplayClear(&TM);
tm1638_TurnOn(&TM, 7);
```

FUNCIÓN	QUÉ HACE
`tm1638_DisplayClear`	Borra la memoria del módulo y lo deja apagado. Al arrancar
`tm1638_TurnOn`	Enciende el display y fija el brillo, de **0 a 7**. El 0 es el más tenue, no es apagado
`tm1638_DisplayTxt`	Escribe un texto en los ocho dígitos, alineado a la derecha
`tm1638_DisplayChar`	Escribe un solo carácter, con su punto decimal si se pide
`tm1638_Seg`	Enciende segmentos sueltos, sin pasar por la tabla de caracteres

FUNCIÓN	QUÉ HACE
`tm1638_Led`	Enciende o apaga uno de los ocho LEDs rojos
`tm1638_ScanButtons`	Lee los ocho pulsadores de una vez: un bit por tecla
`tm1638_KeyState`	Dice si una tecla concreta está pulsada
`tm1638_ReadKey`	Número de la tecla pulsada más baja, o 0 si no hay ninguna

> `tm1638_ScanButtons` devuelve el **nivel**, no el flanco: mientras la tecla siga pulsada, cada llamada volverá a devolver su bit. Detectar la pulsación y filtrar los rebotes es trabajo del programa, y es justamente lo que se aprende en P2.

7.1 Un display de siete segmentos no puede con todas las letras

Es la limitación que más sorprende, y no es de la biblioteca sino del hardware: con siete segmentos hay letras que sencillamente no se pueden dibujar de forma reconocible. Estos son los **38 caracteres** disponibles:

TODO LO QUE SE PUEDE ESCRIBIR

```
[ -0123456789ABCEFHIJLOPSU_abcdhinortuy]
```

NO EXISTEN	QUÉ HACER
Mayúsculas: D G K M N Q R T V W X Y Z	usar la minúscula si existe (`d`, `n`, `r`, `t`, `y`), o buscar otra palabra
Cualquier otro símbolo	solo hay guion y guion bajo

> **No falla en silencio, pero tampoco avisa claro.** Si pides un carácter que no está, la biblioteca borra lo que ibas a mostrar y escribe `Error` seguido de un número. Quien no lo sepa buscará el fallo en su lógica durante un buen rato. Un caso típico: escribir `Go` para dar una salida. **No hay `G` mayúscula**, así que en el display aparece `Error`. Elige otra palabra, por ejemplo `AHOrA`.

8. Trampas que se repiten

Ninguna de estas es hipotética: son las que más tiempo hacen perder. Y ninguna da un mensaje de error que apunte a la causa, que es justamente lo que las hace caras.

SÍNTOMA	CAUSA	PRÁCTICA
El cronómetro atrasa alrededor de un 10 %	> donde debe ir >= al comparar tiempos	P3, P6
Un temporizador da un periodo 5,25 veces mayor del previsto	prescalador calculado para otra frecuencia de reloj	P4
El LED parece encendido fijo en vez de parpadear	el SysTick es de **24 bits**: no admite recargas mayores que 16.777.215	P4
El valor máximo del ADC nunca llega a lo esperado	con 12 bits el máximo es **4095**, no 4096	P5, P6
Un pulsador cuenta dos o tres veces por pulsación	falta el antirrebote	P2 en adelante
Una interrupción salta al soltar el botón, no al pulsarlo	flanco equivocado con un pulsador **activo a cero**	P4, P7
El display pone `Error` y un número	el texto lleva un carácter que no se puede representar	P2 en adelante

SÍNTOMA	CAUSA	PRÁCTICA
Al depurar aparece ensamblador en vez del código en C	la ruta del proyecto lleva tildes o eñes	todas
El código deja de funcionar tras tocar algo en CubeMX	estaba escrito fuera de las marcas `USER CODE`	todas
Funciona en *Debug* y falla en *Release*	falta un `volatile`	P1, P4

8.1 La trampa que no se ve simulando

Una pulsación, en la placa, **no es un evento**: son dos. El flanco que ve la interrupción, y la validación que hace el antirrebote unas decenas de milisegundos más tarde. Cada parte del programa tiene que decidir cuál de los dos le corresponde.

Es fácil escribir código que pase todas las pruebas de escritorio y falle en la mesa, porque al razonar sobre el papel la pulsación se representa como algo instantáneo. El caso más engañoso es cuando dos partes del programa esperan una pulsación seguidas: la primera consume el flanco y la segunda recoge, sin querer, la validación de esa misma pulsación. El programa se salta un paso y no hay forma de verlo leyendo el código.

> **Conclusión práctica: hay una parte de esta asignatura que solo se aprende con la placa delante.** Compilar sin errores y razonar la lógica sobre el papel es necesario, pero no suficiente. Conviene reservar tiempo de laboratorio para probar de verdad lo que se ha escrito.

9. Evaluación

Las siete prácticas siguen el mismo esquema, para que no haya sorpresas al pasar de una a otra.

ELEMENTO	DESCRIPCIÓN
Criterios de aceptación	Seis por práctica, **42 en total**. Cada uno dice qué hay que hacer y qué debe verse. Se comprueban con la placa delante
Tabla de evaluación	Va en el propio enunciado. Reparte la nota entre los criterios
Entregables	El proyecto y una memoria breve. Cada enunciado concreta la extensión

> Los criterios están escritos para ser **observables**: dicen lo que debe verse, no lo que debe sentirse. Si un criterio no se puede comprobar mirando la placa, está mal redactado. Los 42 se recogen juntos, con casillas, en `COMPROBACION EN PLACA.pdf`.

9.1 Consejos para la memoria

- **Mide, no estimes.** «Tarda unos dos segundos» no vale; «medido con cronómetro, 2,1 s en diez repeticiones» sí.
- **Cuenta lo que falló.** Un fallo entendido y explicado vale más que un programa que funcionó a la primera sin saber por qué.
- **Justifica los números.** Cualquier constante del código debería poder explicarse: de dónde sale y qué pasa si se cambia.

10. Qué recibes de cada práctica

En Moovi, cada práctica es una carpeta con dos ficheros:

FICHERO	QUÉ ES
`enunciado.pdf`	Lo que hay que hacer, los criterios de aceptación y cómo se evalúa. **Léelo entero antes de empezar**: los criterios dicen exactamente qué se va a mirar
`ACII_Pn_....zip`	El mismo enunciado más la biblioteca del TM1638, para tenerlo todo junto

Dentro del ZIP, de P2 en adelante:

```
enunciado.pdf
biblioteca/
    TM1638.c
    TM1638.h
LEEME.txt
```

> **La biblioteca se entrega escrita y no hay que modificarla.** Cópiala a tu proyecto tal cual. Si necesitas algo que no hace, se resuelve en tu código, no cambiando la biblioteca: así tu proyecto y el de tus compañeros siguen partiendo de la misma base.

10.1 Qué se entrega

Lo concreta el apartado de entregables de cada enunciado. En general, el proyecto y una memoria breve, por Moovi, en el plazo de la ficha.

> Al comprimir el proyecto, **quita la carpeta `Debug`**. Son decenas de megas de ficheros intermedios que se regeneran al compilar y que no hacen ninguna falta para corregir.